

EKSPLA REMOTECONTROL.DLL — API 参考手册

DLL 版本：v1.6.0.0 及以上 对应头文件： `REMOTECONTROL.H`

错误码

所有函数均返回 `int` 类型错误码，含义如下：

错误码常量	值	含义
<code>RMTCTRLERR_OK</code>	0	成功，无错误
<code>RMTCTRLERR_NOMOREDATA</code>	1	寄存器列表或枚举值列表已到末尾
<code>RMTCTRLERR_NOCFGFILE</code>	2	找不到配置文件
<code>RMTCTRLERR_WRONGCFGFILE</code>	3	配置文件格式错误
<code>RMTCTRLERR_BUFFERTOOSHORT</code>	4	应用程序提供的缓冲区太短
<code>RMTCTRLERR_NOSUCHDEVICE</code>	5	设备名不存在
<code>RMTCTRLERR_NOSUCHREGISTER</code>	6	寄存器名不存在
<code>RMTCTRLERR_CANTCONNECT</code>	7	无法连接
<code>RMTCTRLERR_TIMEOUT</code>	8	等待设备响应超时
<code>RMTCTRLERR_READONLY</code>	9	寄存器只读，不可写入
<code>RMTCTRLERR_NOT_NV</code>	10	寄存器不支持非易失性存储
<code>RMTCTRLERR_HI LIMIT</code>	11	写入值超过上限
<code>RMTCTRLERR_LO LIMIT</code>	12	写入值低于下限
<code>RMTCTRLERR_NOSUCHVALUE</code>	13	写入了不允许的枚举值
<code>RMTCTRLERR_NOTLOGGED</code>	14	该寄存器未被记录（未调用 <code>LogRegStart</code> ）
<code>RMTCTRLERR_MEMORYFULL</code>	15	内存不足

RMTCTRLERR_LOGISEMPTY	16	日志队列中暂无数据
RMTCTRLERR_ALREADYCONNECTED	17	已连接，请先断开
RMTCTRLERR_NOTYETCONNECTED	18	尚未连接，请先连接
RMTCTRLERR_LOGOVERFLOW	0x80000000 0	数据读取成功，但日志 FIFO 已溢出（可与其他错误码按位 OR 叠加）

一、连接函数

rcConnect

```
int rcConnect(int connectiontype, int comportnumber);
```

建立与激光器的连接。

参数	类型	说明
connectiontype	int	连接类型：0 = 直连，1 = RS232
comportnumber	int	COM 口编号：1 =COM1，2 =COM2，...

Python 用法：

```
err = rc.rcConnect(1, 1) # RS232, COM1
```

rcDisconnect

```
int rcDisconnect(void);
```

断开与激光器的连接。无参数。

Python 用法：

```
rc.rcDisconnect()
```

二、设备与寄存器枚举函数

rcGetFirstDeviceName

```
int rcGetFirstDeviceName(char *devname, int maxdevnamelen);
```

获取设备列表中第一个设备的名称。配合 `rcGetNextDeviceName` 循环遍历所有设备。

参数	说明
<code>devname</code>	输出缓冲区，写入设备名字符串
<code>maxdevnamelen</code>	缓冲区最大长度（含结束符）

Python 用法：

```
buf = ctypes.create_string_buffer(256)
err = rc.rcGetFirstDeviceName(buf, 256)
print(buf.value.decode())
```

rcGetNextDeviceName

```
int rcGetNextDeviceName(char *devname, int maxdevnamelen);
```

获取下一个设备名。到列表末尾时返回 `RMTCTRLERR_NOMOREDATA`（值 `1`）。

Python 用法：

```
while True:
    err = rc.rcGetNextDeviceName(buf, 256)
    if err == 1: # NOMOREDATA
        break
    print(buf.value.decode())
```

rcGetFirstRegisterName

```
int rcGetFirstRegisterName(const char *devname, char *regname, int maxregnamelen);
```

获取指定设备的第一个寄存器名称。

参数	说明
devname	设备名（如 <code>b"NL30x:8"</code> ）
regname	输出缓冲区，写入寄存器名
maxregnamelen	缓冲区最大长度

rcGetNextRegisterName

```
int rcGetNextRegisterName(char *regname, int maxparlen);
```

获取下一个寄存器名。到末尾时返回 `RMTCTRLERR_NOMOREDATA`。

Python 枚举所有寄存器示例：

```
dev = b"NL30x:8"
buf = ctypes.create_string_buffer(256)
err = rc.rcGetFirstRegisterName(dev, buf, 256)
while err == 0:
    print(buf.value.decode())
    err = rc.rcGetNextRegisterName(buf, 256)
```

三、寄存器属性查询函数

rcIsRegisterWriteable

```
int rcIsRegisterWriteable(const char *devname, const char *regname, int *isWriteable);
```

查询寄存器是否可写。

输出参数 <code>*isWriteable</code>	含义
--------------------------------	----

1	可写
0	只读

Python 用法：

```
is_writable = ctypes.c_int()
err = rc.rcIsRegisterWriteable(b"NL30x:8", b"State", ctypes.byref(is_writable))
print(is_writable.value) # 1 = 可写
```

rcIsRegisterNV

```
int rcIsRegisterNV(const char *devname, const char *regname, int *isNV);
```

查询寄存器是否支持非易失性存储（掉电保存）。

输出参数 <code>*isNV</code>	含义
1	支持 NV（掉电保存）
0	不支持 NV（掉电丢失）

Python 用法：

```
is_nv = ctypes.c_int()
err = rc.rcIsRegisterNV(b"NL30x:8", b"Sync mode", ctypes.byref(is_nv))
print(is_nv.value) # 1 = 支持NV
```

rcGetRegFirstEnumValue / rcGetRegNextEnumValue

```
int rcGetRegFirstEnumValue(const char *devname, const char *regname,
                           char *enumname, int maxenumnamelen);
int rcGetRegNextEnumValue(const char *devname, const char *regname,
                           char *enumname, int maxenumnamelen);
```

枚举寄存器的所有合法值（仅对枚举类型寄存器有效，数值型返回 `RMTCTRLERR_NOMOREDATA`）。

Python 枚举合法值示例：

```

dev = b"NL30x:8"
reg = b"State"
buf = ctypes.create_string_buffer(64)
err = rc.rcGetRegFirstEnumValue(dev, reg, buf, 64)
while err == 0:
    print(buf.value.decode())    # 依次输出: SLEEP, STOP, RUN, FAULT
    err = rc.rcGetRegNextEnumValue(dev, reg, buf, 64)

```

四、寄存器读写函数

rcGetRegAsDouble — 读取浮点值

```

int rcGetRegAsDouble(const char *devname, const char *regname,
                    double *value, int timeout, int *timestamp);

```

参数	说明
value	输出，读取到的 double 值
timeout	超时毫秒数，-1 = 无限等待
timestamp	输出，消息时间戳（不需要可传 NULL）

Python 用法：

```

val = ctypes.c_double()
ts = ctypes.c_int()
err = rc.rcGetRegAsDouble(b"NL30x:8", b"Repetition Rate",
                        ctypes.byref(val), 2000, ctypes.byref(ts))
print(val.value)

```

rcGetRegAsString — 读取字符串值

```

int rcGetRegAsString(const char *devname, const char *regname,
                    char *value, int maxvallen, int timeout, int *timestamp);

```

参数	说明
value	输出缓冲区

maxvallen	缓冲区最大长度（含结束符）
timeout	超时毫秒数， -1 = 无限等待
timestamp	输出，消息时间戳（不需要可传 NULL）

Python 用法：

```
buf = ctypes.create_string_buffer(256)
ts = ctypes.c_int()
err = rc.rcGetRegAsString(b"NL30x:8", b"State",
                           buf, 256, 2000, ctypes.byref(ts))
print(buf.value.decode()) # e.g. "STOP"
```

rcSetRegFromDouble — 写入浮点值（掉电不保存）

```
int rcSetRegFromDouble(const char *devname, const char *regname, double value);
```

Python 用法：

```
err = rc.rcSetRegFromDouble(b"MaxiOPG:31",
                             "ATTN. RotH 1.8°/64 600mA.".encode("latin1"),
                             ctypes.c_double(272))
```

rcSetRegFromDoubleA — 写入浮点值（带强制限制选项）

```
int rcSetRegFromDoubleA(const char *devname, const char *regname,
                         double value, int forcelimits);
```

比 rcSetRegFromDouble 多一个参数：

forcelimits	行为
0	值超限时报错（HILIMIT / LOLIMIT）
非0	值超限时自动截断到边界值

rcSetRegNVFromDouble — 写入浮点值（掉电保存）

```
int rcSetRegNVFromDouble(const char *devname, const char *regname, double value);
```

与 `rcSetRegFromDouble` 相同，但写入结果掉电后仍保留。仅对 `Nonvolatile = True` 的寄存器有效，否则返回 `RMTCTRLERR_NOT_NV`（值 10）。

rcSetRegFromString — 写入字符串值（掉电不保存）

```
int rcSetRegFromString(const char *devname, const char *regname, const char *value);
```

Python 用法：

```
err = rc.rcSetRegFromString(b"NL30x:8", b"State", b"RUN")
```

rcSetRegFromStringA — 写入字符串值（带强制限制选项）

```
int rcSetRegFromStringA(const char *devname, const char *regname,
                        const char *value, int forcelimits);
```

与 `rcSetRegFromDoubleA` 类似，多一个 `forcelimits` 参数控制越界行为。

rcSetRegNVFromString — 写入字符串值（掉电保存）

```
int rcSetRegNVFromString(const char *devname, const char *regname, const char *value);
```

与 `rcSetRegFromString` 相同，但写入结果掉电后仍保留。

五、寄存器日志函数

用于后台持续记录某个寄存器的变化，存入内存队列，然后按需取出。

rcLogRegStart — 开始记录

```
int rcLogRegStart(const char *devname, const char *regname, int queuesize);
```

参数	说明
<code>queuesize</code>	为日志队列分配的内存大小（字节）

Python 用法：

```
err = rc.rcLogRegStart(b"NL30x:8", b"Pump voltage", 1024)
```

rcLogRegStop — 停止记录

```
int rcLogRegStop(const char *devname, const char *regname);
```

rcGetRegFromLogAsDouble — 从日志队列读取浮点值

```
int rcGetRegFromLogAsDouble(const char *devname, const char *regname,  
                             double *value, int *timestamp);
```

从日志 FIFO 队列中取出一条记录（先进先出）。队列为空时返回 `RMTCTRLERR_LOGISEMPTY`。若读取成功但队列已溢出，返回值会额外叠加 `RMTCTRLERR_LOGOVERFLOW`（`0x80000000`）。

Python 连续读取示例：

```
val = ctypes.c_double()  
ts = ctypes.c_int()  
while True:  
    err = rc.rcGetRegFromLogAsDouble(b"NL30x:8", b"Pump voltage",  
                                     ctypes.byref(val), ctypes.byref(ts))  
  
    if err == 16: # LOGISEMPTY  
        break  
    print(f"t={ts.value} v={val.value}")
```

rcGetRegFromLogAsString — 从日志队列读取字符串值

```
int rcGetRegFromLogAsString(const char *devname, const char *regname,  
                             char *value, int maxvallen, int *timestamp);
```

六、函数速查表

函数	类别	说明
rcConnect	连接	建立连接
rcDisconnect	连接	断开连接
rcGetFirstDeviceName	枚举	获取第一个设备名
rcGetNextDeviceName	枚举	获取下一个设备名
rcGetFirstRegisterName	枚举	获取设备第一个寄存器名
rcGetNextRegisterName	枚举	获取设备下一个寄存器名
rcIsRegisterWriteable	属性查询	查询寄存器是否可写
rcIsRegisterNV	属性查询	查询寄存器是否支持 NV
rcGetRegFirstEnumValue	属性查询	获取第一个合法枚举值
rcGetRegNextEnumValue	属性查询	获取下一个合法枚举值
rcGetRegAsDouble	读	读取寄存器浮点值
rcGetRegAsString	读	读取寄存器字符串值
rcSetRegFromDouble	写 (volatile)	写入浮点值，掉电丢失
rcSetRegFromDoubleA	写 (volatile)	写入浮点值，支持强制限制
rcSetRegNVFromDouble	写 (NV)	写入浮点值，掉电保留
rcSetRegFromString	写 (volatile)	写入字符串值，掉电丢失
rcSetRegFromStringA	写 (volatile)	写入字符串值，支持强制限制
rcSetRegNVFromString	写 (NV)	写入字符串值，掉电保留
rcLogRegStart	日志	开始记录寄存器变化
rcLogRegStop	日志	停止记录
rcGetRegFromLogAsDouble	日志	从队列取浮点值

rcGetRegFromLogAsString	日志	从队列取字符串值
-------------------------	----	----------